

Laporan Pembuatan Project UAS IOT Sederhana Smarthome

Sistem Mikroprosesor

Dosen Pengampu : Muhammad Ikhwan Fathulloh, S.Kom.



Di buat oleh:

Nama : Muhamad Farhan Rizki

Nim : 23552011416

Kelas : TIFK-23A

FAKULTAS INDUSTRI KREATIF

DEPARTEMEN TEKNIK INFORMATIKA

UNIVERSITAS TEKNOLOGI BANDUNG

2026

1. Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi Internet of Things (IoT) memungkinkan berbagai perangkat elektronik saling terhubung melalui jaringan internet. Salah satu penerapan IoT yang banyak dikembangkan adalah Smart Home, yaitu sistem otomatisasi rumah yang memungkinkan pengguna mengontrol dan memantau perangkat rumah tangga dari jarak jauh.

Pada sistem konvensional, pengendalian lampu masih dilakukan secara manual menggunakan saklar. Hal ini kurang efisien, terutama ketika pengguna ingin mengontrol perangkat dari lokasi yang berbeda. Oleh karena itu, dibutuhkan sistem Smart Home berbasis IoT yang mampu mengatasi permasalahan tersebut.

Proyek ini merancang sistem Smart Home menggunakan ESP32 sebagai mikrokontroler utama, Web Server sebagai antarmuka pengguna, MQTT sebagai protokol komunikasi data, serta FreeRTOS untuk mengatur multitasking secara paralel.

2. Tujuan

Tujuan dari pembuatan sistem ini adalah:

- Merancang sistem Smart Home berbasis IoT menggunakan ESP32.
- Mengimplementasikan kontrol lampu melalui web dashboard.
- Mengintegrasikan protokol MQTT untuk komunikasi dua arah.
- Menggunakan FreeRTOS agar sistem dapat berjalan secara multitasking dan non-blocking.

3. Manfaat

Manfaat dari sistem ini antara lain:

- Memudahkan pengguna dalam mengontrol lampu dari jarak jauh.
- Memberikan pemantauan status lampu secara real-time.

- Menjadi media pembelajaran implementasi IoT, MQTT, dan FreeRTOS.
- Sistem lebih stabil karena tidak menggunakan delay blocking.

4. Arsitektur Sistem

Sistem terdiri dari beberapa komponen utama:

- ESP32: Sebagai pengendali utama sistem.
- Web Dashboard: Antarmuka pengguna berbasis web.
- Broker MQTT (public.cloud.shiftr.io): Media komunikasi publish-subscribe.
- FreeRTOS: Mengatur task MQTT dan LED secara paralel.

5. Struktur Program

Struktur kode program dibagi menjadi beberapa bagian utama:

- Inisialisasi library dan pin
- Konfigurasi MQTT
- Variabel global dan pointer
- Interrupt
- Callback MQTT
- Task FreeRTOS
- Web Server dan API
- Manajemen WiFi

6. Penggunaan Millis()

Pada sistem embedded berbasis mikrokontroler seperti ESP32, penggunaan delay() secara berlebihan dapat menyebabkan sistem menjadi blocking, yaitu proses lain tidak dapat berjalan selama waktu tunggu tersebut. Hal ini sangat tidak disarankan, terutama pada sistem IoT yang membutuhkan koneksi jaringan, web server, dan komunikasi MQTT secara bersamaan.

Fungsi millis() digunakan untuk mengelola waktu secara non-blocking, yaitu menghitung selisih waktu sejak ESP32 menyala tanpa menghentikan proses utama. Dalam program ini, millis() digunakan pada proses koneksi WiFi untuk memberikan batas waktu (timeout) koneksi.

Contoh dalam kode:

```
unsigned long t = millis();

while(WiFi.status()!=WL_CONNECTED && millis()-t < 15000){

    delay(300);
```

```
}
```

Penjelasan:

- Variabel `t` menyimpan waktu awal saat proses koneksi dimulai.
- Program akan terus mencoba koneksi WiFi selama maksimal 15 detik.
- Jika WiFi gagal terhubung, sistem tidak akan hang, melainkan berpindah ke mode Access Point (AP).
- Pendekatan ini memastikan sistem tetap stabil dan dapat dikonfigurasi ulang melalui web.

Dengan penggunaan `millis()`, sistem tetap responsif, mendukung multitasking, dan sesuai dengan prinsip manajemen waktu pada sistem mikroprosesor.

7. Penggunaan Interrupt()

Interrupt digunakan untuk menangani input dari tombol fisik secara real-time tanpa perlu melakukan polling di dalam loop utama. Hal ini sangat penting agar sistem tetap efisien dan cepat merespons input pengguna.

Pada program ini, interrupt digunakan untuk mengontrol Lampu Kamar melalui tombol fisik yang terhubung ke pin `BTN_PIN`.

Contoh dalam kode:

```
void IRAM_ATTR toggleLampuKamar() {  
  
    *lampuKamar = !(*lampuKamar);  
  
    Serial.println("[INTERRUPT] Toggle Lampu Kamar");  
  
}
```

Penjelasan:

- Fungsi `IRAM_ATTR` memastikan fungsi interrupt disimpan di memori internal agar dapat dieksekusi dengan cepat.
- Interrupt dipicu dengan mode `FALLING`, yaitu saat tombol ditekan.
- Status lampu diubah langsung tanpa menunggu eksekusi loop utama.
- Hal ini membuktikan bahwa sistem tidak menggunakan polling, sesuai dengan konsep interrupt pada mikroprosesor.

Penggunaan interrupt ini meningkatkan responsivitas sistem.

8. Penggunaan Pointer()

Pointer digunakan untuk mengakses dan memodifikasi variabel status lampu secara efisien, terutama ketika variabel tersebut digunakan oleh banyak bagian program seperti interrupt, task FreeRTOS, MQTT callback, dan web server.

Contoh dalam kode:

```
bool lampuKamarState = false;

bool lampuTamuState = false;

bool *lampuKamar = &lampuKamarState;

bool *lampuTamu = &lampuTamuState;
```

Penjelasan:

- Pointer menyimpan alamat memori dari variabel status lampu.
- Dengan pointer, perubahan status lampu dari satu fungsi akan langsung terbaca oleh fungsi lain.
- Hal ini penting untuk menjaga sinkronisasi data antar task FreeRTOS dan interrupt.
- Pointer juga mencerminkan penerapan manajemen memori sederhana pada sistem mikroprosesor.

Penggunaan pointer ini sudah tepat dan relevan dengan materi sistem mikroprosesor.

9. Penggunaan MQTT (Message Queuing Telemetry Transport)

MQTT digunakan sebagai protokol komunikasi data antara ESP32, web dashboard, dan broker MQTT. MQTT menerapkan konsep publish-subscribe, sehingga komunikasi data menjadi ringan dan efisien.

9.1 Topic

Topic yang digunakan:

- smarthome/esp32/lampu/kamar
- smarthome/esp32/lampu/tamu

Topic ini berfungsi sebagai jalur komunikasi logis antara perangkat dan dashboard.

9.2 Payload

Payload yang digunakan pada sistem ini bersifat sederhana, yaitu:

- ON
- OFF

Payload tersebut berfungsi sebagai perintah logika untuk menentukan status lampu, bukan nilai intensitas secara langsung.

Ketika payload ON diterima, sistem akan mengubah status lampu menjadi aktif (true). Selanjutnya, pada bagian kontrol output LED, status ini diterjemahkan menjadi sinyal PWM dengan duty cycle tertentu untuk menyalakan LED. Sebaliknya, payload OFF akan mematikan lampu dengan mengatur duty cycle PWM menjadi nol.

Dengan pendekatan ini, sistem tetap mudah digunakan, namun lebih fleksibel karena PWM dapat diatur tanpa mengubah format payload MQTT.

9.3 Callback MQTT

Callback MQTT berfungsi untuk memproses pesan yang diterima dari broker.

Contoh dalam kode:

```
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    if (String(topic) == "smarthome/esp32/lampu/kamar") {
        *lampuKamar = (msg == "ON");
    }
}
```

Penjelasan:

- Fungsi ini otomatis dipanggil ketika ESP32 menerima pesan MQTT.
- Pesan yang diterima langsung mempengaruhi status lampu.
- Sistem menjadi sinkron antara web dashboard, serial monitor, dan broker MQTT.

Penggunaan MQTT ini sudah sesuai dengan standar komunikasi IoT.

9.4 Implementasi PMW pada Output LED

Pulse Width Modulation (PWM) digunakan untuk mengontrol tingkat kecerahan LED pada ESP32. PWM bekerja dengan mengatur duty cycle, yaitu persentase waktu sinyal berada pada kondisi HIGH dalam satu periode.

Pada sistem ini, PWM diimplementasikan menggunakan modul LEDC (LED Control) bawaan ESP32.

Contoh implementasi PWM pada task LED:

```
int duty = (*lampuKamar || *lampuTamu) ?
```

```
0 : 255;
```

```
ledcWrite(LED_CHANNEL, 0);
```

Pada sistem ini, PWM digunakan untuk mengatur tingkat kecerahan LED menggunakan modul LEDC pada ESP32. Nilai duty cycle menentukan seberapa lama sinyal berada pada kondisi HIGH dalam satu periode.

- Jika salah satu lampu (Lampu Kamar atau Lampu Tamu) dalam kondisi aktif, maka nilai duty cycle diatur ke 180 dari maksimum 255 (sekitar 70%). Hal ini membuat LED menyala dengan intensitas sedang.
- Jika kedua lampu dalam kondisi mati, maka nilai duty cycle diatur ke 0, sehingga LED padam sepenuhnya.

Dengan pendekatan ini, LED tidak hanya berfungsi sebagai indikator ON/OFF, tetapi juga menunjukkan kondisi sistem menggunakan variasi intensitas cahaya.

10. Penggunaan FreeRTOS

ESP32 memiliki sistem operasi real-time (FreeRTOS) yang memungkinkan beberapa proses berjalan secara bersamaan (multitasking). Pada sistem ini, FreeRTOS digunakan untuk memisahkan tugas agar tidak saling mengganggu.

10.1 Task MQTT

```
xTaskCreatePinnedToCore(taskMQTT,"MQTT",4096,NULL,1,NULL,1);
```

Tugas ini:

- Menjaga koneksi MQTT tetap aktif.
- Menjalankan mqttClient.loop() secara terus-menerus.
- Memastikan komunikasi MQTT berjalan stabil tanpa mengganggu web server.

10.2 Task LED

```
xTaskCreatePinnedToCore(taskLED,"LED",2048,NULL,1,NULL,1);
```

Tugas ini:

- Mengontrol LED berdasarkan status lampu.
- Berjalan secara paralel tanpa memblokir proses lain.
- Membuat sistem lebih rapi dan terstruktur.

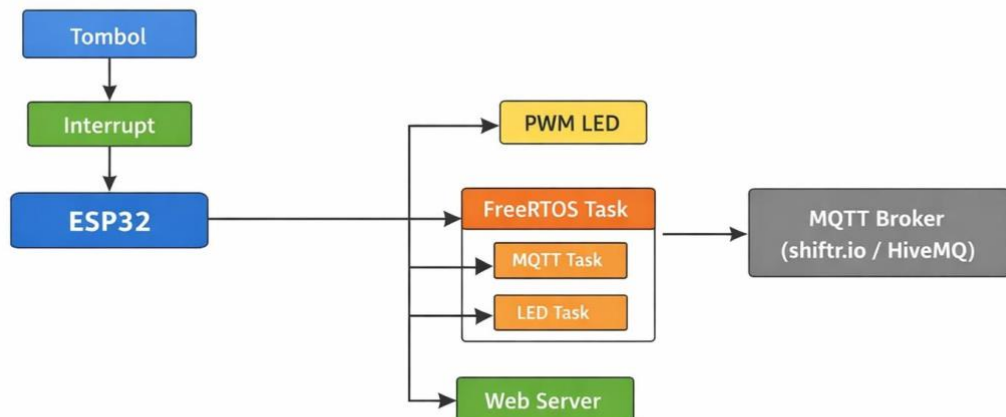
Dengan FreeRTOS, sistem menjadi non-blocking, stabil, dan sesuai dengan konsep multitasking mikroprosesor.

11. Alur Sistem Dan Diagram Blok

11.1 Alur Sistem

Sistem dimulai dengan ESP32 melakukan pengecekan konfigurasi WiFi. Jika data WiFi belum tersedia, ESP32 akan masuk ke mode Access Point (ESP32-Config) untuk konfigurasi melalui web dashboard. Setelah terhubung ke WiFi, sistem menjalankan web server, MQTT client, dan task FreeRTOS secara paralel. Input dari tombol diproses menggunakan interrupt, sedangkan perintah dari web dan MQTT diproses melalui callback. Status lampu dikontrol menggunakan PWM dan disimpan dalam variabel global yang diakses menggunakan pointer.

11.2 Diagram Blok



12. Pengujian

1. Kontrol lampu melalui web berhasil.
2. Publish dan subscribe MQTT berjalan dengan baik.
3. Status lampu sinkron antara web, serial monitor, dan broker MQTT.
4. Sistem tetap responsif berkat FreeRTOS.

13. Kesimpulan

Berdasarkan hasil perancangan dan pengujian, dapat disimpulkan bahwa sistem Smart Home berbasis ESP32 ini berhasil diimplementasikan dengan baik. Integrasi Web Server, MQTT, dan FreeRTOS membuat sistem lebih efisien, responsif, dan real-time.

Sistem ini dapat dikembangkan lebih lanjut dengan menambahkan sensor lain seperti suhu, kelembaban, atau keamanan rumah.

Lampiran

1. ***Link Demo YouTube:***

<https://youtu.be/UvEgn4nhbfk?si=5wGZxj86LK0gALg5>

2. ***Link Github:***

https://github.com/MuhamadFarhanRizki/Smarthome_IOT_Muhamad_Farhan_Rizki_23552011416.git